

## WEEK 10 · REFERENCE

# THE ONE-WAY LOOP, IN DEPTH

Every demo of the old lecture, the catalogue of what each layer may never do, and five measured answers to questions the cycles raise and do not stop for.

---

## WHAT IS IN HERE

Eight demos that compute their own claims, components and escaping, identity, derivation, and performance measured on this machine.

---

## HOW TO READ IT

Each slide is the depth behind one cycle. The chip in the corner says which spine slide points here.

---

## WHEN TO OPEN IT

Mid-refactor, when a rule has turned into a guess — or before week 13, to see every mistake once more.

► **OPTIONAL DEPTH, NOT EXAMINED.** Everything the assignment and the exam need was typed in a cycle.

# TWO COPIES, AND THE DISTANCE BETWEEN THEM

left: the array · right: the rows · far right: how far apart, computed↶ reloadopen ↷

ONE LIST, TWO PLACES IT IS WRITTEN DOWN

push to the array only

append a row only

remove a row only

compare the two

redraw from the array

reset

THE ARRAY — ITEMS

```
[  
  "flour",  
  "olive oil",  
  "almond milk"  
]
```

THE SCREEN — #LIST

flour

olive oil

almond milk

HOW FAR APART THEY ARE

0

the two agree

Nothing on this page repairs the drift, and that is the point: every repair you can think of is one more line that one more handler has to remember, forever.

► **PRESS "APPEND A ROW ONLY" THREE TIMES, THEN ASK WHAT THE ARRAY HOLDS.** Vote before you press compare.

# THE SAME APPLICATION, TWICE

left: patched · right: redrawn · identical data, identical buttons

↻ reload open ↗

THE SAME APPLICATION, PATCHED ON THE LEFT AND REDRAWN ON THE RIGHT

add one remove the first +1 on the first sort by quantity 20 random operations, both panels redraw both from their data reset

PATCHED — EVERY HANDLER DRAWS

flour2

olive oil1

almond milk4

cheese1

30 lines across four handlers

agrees with its data

REDRAWN — EVERY HANDLER ONLY CHANGES DATA

flour2

olive oil1

almond milk4

cheese1

20 lines across four handlers

agrees with its data

The left panel is not written badly. Every handler in it is correct on its own, and it was tested one button at a time. What it cannot survive is the two hundred orderings of those buttons that nobody tested.

► Press each button once and both agree. Then **20 RANDOM OPERATIONS** — and then “redraw both”.

← SPINE · 13

3

## “SO THIS IS REACT?”

### WHAT REACT ADDS

A description of the screen you return instead of draw, and a diff so that redrawing everything only **writes** what changed.

### WHAT REACT DOES NOT ADD

The model. **One state, one render, one direction** is the idea React was built to make cheap — it did not invent it.

### WHY WE DO IT BY HAND

Because the part that is hard to learn is the model, and the part that is easy to learn is the library. Doing it once by hand makes the second one a Tuesday.

**You are learning React thinking. The import statement is next semester's problem.**

# A FUNCTION, NOT AN OBJECT LITERAL

SHARED BY ACCIDENT

```
1 export const INITIAL = {  
2   items: [],  
3   query: '',  
4 };
```

One object. Every page that imports it gets **the same one**, and one mutation leaks into the next.

FRESH EVERY TIME

```
1 function initialState() {  
2   return {  
3     items: [],  
4     query: '',  
5   };  
6 }
```

Called once per load. **Week 8's value-versus-reference**, arriving where it costs something.

## SEED() COPIES — ONE LEVEL DEEP

WHAT CYCLE 1 TYPES

```
1 books.map((book) => ({ ...book })))
```

A new array of new objects. The seed file and the state no longer share an object — enough, because every field of a book is a string or a number.

WHEN A FIELD IS ITSELF AN OBJECT

```
1 const seed = [{ id: 'b1', tags: ['x'] }];  
2 const copy = seed.map((b) => ({ ...b }));  
3 copy[0].tags.push('y');  
4 seed[0].tags; // ['x', 'y'] — shared
```

Spread copies one level. `structuredClone(seed)` copies all the way down — measured: a push into the clone leaves the seed untouched.

LIVE CODING · 6 MINUTES

# TAKE THE REMOVE HANDLER OUT OF THE MONOLITH

- 1 Open the week-9 `app.js` on the projector.  
**Do not read it first.**
- 2 Create `js/state.js` with `getState`, `setState`, `subscribe`. Move `items` into it.
- 3 Move the remove handler into `js/events.js`, and delete every line in it that touches an element.
- 4 **Run it.**  
The page is blank. Ask the room why before you fix it.

► Then add line 4 of `app.js`, and it works.

# RENDER(STATE) IS A FUNCTION OF ITS ARGUMENT

edit the object on the left, press render, and there is no third input

reload

open

THE SCREEN IS A FUNCTION OF THIS OBJECT, AND OF NOTHING ELSE

render(state)

render twice, compare the two

delete a row straight from the DOM

subscribe without the first draw

reset

STATE — EDIT IT

the whole application, as one object

```
{
  "items": [
    {
      "id": "a1",
      "name": "flour",
      "qty": 2
    },
    {
      "id": "a2",
      "name": "olive oil",

```

valid JSON

THE SCREEN — RENDER(STATE)

showing 3 of 3 · renders so far: 1

almond milk 4

flour 2

olive oil 1

nothing measured yet

There is no button here that changes the screen without changing the object. That is not a limitation of the demo — it is the whole rule, and everything else this week follows from it.

► **“DELETE A ROW STRAIGHT FROM THE DOM” — IS THE ITEM GONE?** Take the vote. Then press render.

# THREE THINGS RENDER MAY NEVER DO, AND WHY EACH ONE BITES

## FORBIDDEN

## WHAT HAPPENS IF YOU DO

change state

it calls itself. You debug a frozen tab.

add a listener

one more listener per redraw. After ten keystrokes the handler runs ten times.

read the DOM to decide

the screen is now a source of truth, and law 1 is gone.

► The second one is invisible everywhere except the Event Listeners panel — and in how many times your `console.log` prints.

# WHY A LISTENER INSIDE RENDER MULTIPLIES — AND WHEN IT WOULD NOT

AN ARROW, TEN TIMES

```
1 for (let i = 0; i < 10; i++) {  
2   button.addEventListener('click', () => { n += 1; });  
3 } // one click: n === 10
```

THE SAME FUNCTION, TEN TIMES

```
1 function onClick() { n += 1; }  
2 for (let i = 0; i < 10; i++) {  
3   button.addEventListener('click', onClick);  
4 } // one click: n === 1
```

► The DOM refuses to add **THE SAME FUNCTION** twice for the same type — and an arrow written inside `render` is a **NEW** function on every draw. The fix is not a named function: it is the listener in `events.js`, registered once.

# A COMPONENT IS A FUNCTION: ITEM IN, ELEMENT OUT

```
1  const badge = (text) => `<span class="qty">${esc(text)}</span>`;
2
3  const row = (item) => `<li data-id="${esc(item.id)}">
4    <span class="item-title">${esc(item.title)}</span>
5    ${badge(item.number)}
6  </li>`;
7
8  const list = (items) => `<ul id="list">${items.map(row).join('')}</ul>`;
```

► No side effects, no knowledge of where the result will be put, and `list` calls `row` calls `badge`. That is composition, with no framework in the room.

# THE PRICE OF A TEMPLATE LITERAL IS SIX LINES

the same row four ways · identical markup · and one item with a hostile name

reload

open

A COMPONENT IS A FUNCTION: ITEM IN, MARKUP OUT

build the same row four ways

compose: list calls row calls badge

give one item a hostile name

reset

## 1 · STRING CONCATENATION

```
function concatRow(item) {
  return (
    '<li data-id="' +
    item.id +
    '>' +
    '<span class="name">' +
    item.name +
    '</span>' +
    '<span class="qty">' +
    item.qty +
    '</span>' +
    '</li>'
  );
}
```

flour

2

## 2 · TEMPLATE LITERAL, UNESCAPED

```
function rawRow(item) {
  return `<li data-id="${item.id}">
  <span class="name">${item.name}</span>
  <span class="qty">${item.qty}</span>
</li>`;
}
```

flour

2

## 3 · TEMPLATE LITERAL, ESCAPED

```
function safeRow(item) {
  return `<li data-id="${esc(item.id)}">
  <span class="name">${esc(item.name)}</span>
  <span class="qty">${esc(item.qty)}</span>
</li>`;
}
```

flour

2

## 4 · CREEATEELEMENT

```
function domRow(item) {
  const li = document.createElement('li');
  li.dataset.id = item.id;
  const name = document.createElement('span');
  name.className = 'name';
  name.textContent = item.name;
  const qty = document.createElement('span');
  qty.className = 'qty';
  qty.textContent = item.qty;
  li.append(name, qty);
  return li;
}
```

flour

2

ARE THE FOUR OUTPUTS THE SAME?

all four produced identical markup

TIMES THE ITEM'S NAME EXECUTED

0

The hostile name is an onerror attribute on an image that cannot load. A `<script>` tag inserted as markup does not run, which is exactly why it is the

COMPOSITION

| WAY                  | SOURCE LINES | THE ATTACKER'S CODE RAN |
|----------------------|--------------|-------------------------|
| string concatenation | 14           | no                      |

► **PRESS "HOSTILE NAME" FIRST.** Two of the four execute it. Which two?

← SPINE · 13

## SIX LINES, AND THEY ARE THE WHOLE PRICE

```
1 function esc(value) {  
2   return String(value)  
3     .replaceAll('&', '&amp;')  
4     .replaceAll('<', '&lt;')  
5     .replaceAll('>', '&gt;')  
6     .replaceAll('"', '&quot;');  
7 }
```

► & **FIRST**, or you escape your own escapes. And `String(value)` first, because a number has no `replaceAll`.

# WHEN TO USE WHICH, AND IT IS NOT A MATTER OF TASTE

## TEMPLATE LITERAL + `esc`

A component you want to READ.  
Shape is visible; the whole row fits on the screen.

## `createElement`

A row that needs an element reference, a `dataset`, or a listener attached to it directly.

## EITHER ONE WITHOUT `esc`

An item name is a string a user typed. It reaches the HTML parser, and it is the shortest wrong answer in the week.

► This assignment uses `createElement`, because your rows carry a `data-id` and the act after this one is about that.

# NOTHING IS WRONG UNTIL SOMETHING REORDERS THE LIST

the same six items · left keyed on position · right keyed on id reload open

TWO LISTS, ONE KEYED ON WHERE A ROW SITS AND ONE ON WHAT IT IS

sort both by quantity search both for "il" clear the search **+1 on the third row of each** remove the third row of each redraw both reset

KEYED ON POSITION — DATA-INDEX

|             |   |
|-------------|---|
| flour       | 2 |
| olive oil   | 1 |
| almond milk | 4 |
| cheese      | 1 |
| paprika     | 3 |
| oat milk    | 2 |

nothing pressed yet

```
function bumpByIndex(row) {  
  /* The row's position in the list ON SCREEN, used as a p  
    COLLECTION. Those are the same number only while noth  
  const item = A.items[Number(row.dataset.index)];  
  if (!item) return null;  
  item.qty += 1;  
  row.querySelector('.qty').textContent = item.qty;  
  return item;  
}
```

KEYED ON IDENTITY — DATA-ID

|             |   |
|-------------|---|
| flour       | 2 |
| olive oil   | 1 |
| almond milk | 4 |
| cheese      | 1 |
| paprika     | 3 |
| oat milk    | 2 |

nothing pressed yet

```
function bumpById(row) {  
  const item = B.items.find((candidate) => candidate.id ==  
  if (!item) return null;  
  item.qty += 1;  
  row.querySelector('.qty').textContent = item.qty;  
  return item;  
}
```

TIMES THE WRONG ITEM CHANGED

0

the two collections agree

Both lists start identical and stay identical while nothing reorders them. Sort first, then press a row button, and count.

► **PRESS +1 BEFORE YOU SORT.** The counter stays at zero. Now sort, and press it again.

# ONE LISTENER, ON SOMETHING RENDER NEVER REPLACES

```
1 list.addEventListener('click', (event) => {  
2   const row = event.target.closest('li[data-id]');  
3   if (!row) return;  
4   // ... setState from row.dataset.id  
5 });
```

► Week 9 made delegation a good habit. **THIS WEEK IT STOPS BEING OPTIONAL:** before today you chose when to redraw; from today it happens on every keystroke.

# HOW LONG A STORED ANSWER STAYS TRUE

four questions about one collection · stored on the left, derived on the right

reload open

FOUR QUESTIONS ABOUT ONE COLLECTION, STORED ON THE LEFT AND DERIVED ON THE RIGHT

add an item · updates everything remove one · updates everything quick fix: bulk import · updates the collection only rename a shelf · updates the collection only reset

STORED — FOUR FIELDS BESIDE THE COLLECTION

| QUESTION | THE STORED ANSWER | ASKED AFRESH |
|----------|-------------------|--------------|
| count    | 3                 | 3            |
| total    | 7                 | 7            |
| shelves  | cold, dry         | cold, dry    |
| heaviest | almond milk       | almond milk  |

nothing changed yet

DERIVED — FOUR FUNCTIONS, NO FIELDS

count(items)3

total(items)7

shelves(items)cold, dry

heaviest(items)almond milk

Nothing here can go out of date, and the reason is not discipline. There is nothing here to go out of date.

STORED ANSWERS CURRENTLY LYING

0

the two panels agree

answering all four afresh, averaged over 1000 runs: 0.0006 ms. Drawing them once: 0.000 ms. Act 9 is where those numbers start to matter.

► The first two buttons keep every stored field correct.

**THE THIRD ONE IS WRITTEN A MONTH LATER, BY SOMEBODY WHO ONLY NEEDED TO CHANGE THE COLLECTION.**

# WHAT SELECT.VALUE DOES WHEN THE OPTION IS GONE

## WHAT YOU DO

## WHAT THE SELECT SAYS

replace its options while `x` is chosen    `value` is the first option; `selectedIndex` 0

assign a value no option has    `value` is `''`; `selectedIndex` -1

then `if (select.value === '')` “all” the `selectedIndex` type is

- ▶ The honest limit: remove the last book of the chosen genre and the select falls back to “all” while `state.genre` still names the gone genre — the list is empty under a filter that says all. **THE FIX BELONGS IN STATE**, not in the select.

# SORTING A COPY, AND COMPARING LIKE A PERSON

A COPY, THREE WAYS

```
1 books.filter(match).sort(byTitle); // filter already made a new array
2 [...books].sort(byTitle);          // spread makes one
3 books.toSorted(byTitle);           // ES2023: never mutates
```

THE COMPARISON

```
1 ['b', 'a', 'B', 'A'].sort(); // A, B, a, b
2 ['b', 'a', 'B', 'A'].sort((x, y) => x.localeCompare(y, 'en')); // a, A, b, B
```

► The default compares UTF-16 code units, so every capital comes before every lower-case letter. `localeCompare` with a locale is the order a person expects — cycle 3 passes `'he'`. `toSorted` is Baseline, widely available since July 2023 (MDN).

## THE THIRD PAGE HAS NO DATA, AND THAT IS THE PROOF

```
1 export function render(state) {  
2   renderList(state);    // returns immediately if #list is not on this page  
3   renderDetail(state);  
4   renderSummary(state); // returns immediately if #summary-count is not here  
5 }
```

► One entry point, three pages, and no `if` anywhere that asks which page it is on. **ADDING A FOURTH PAGE IS A MOUNT AND A FUNCTION.**

# THE ID IN THE ADDRESS: GET() AND ENCODEURIComponent()

| THE ADDRESS     | new URLSearchParams(location.search).get('id') |
|-----------------|------------------------------------------------|
| book.html?id=b4 | 'b4'                                           |
| book.html?id=   | ' ' — found, and empty                         |
| book.html       | null — not found                               |

► Writing the link: `encodeURIComponent('a b&c=d')` is `a%20b%26c%3Dd` — an id a user made can hold any character, and `&` or `=` would split it. `get()` decodes it back.

# WHY THE SPLIT MAKES EVERY FUTURE CHANGE LOCAL

the same 47 lines · as one file and as three · pick a change request

reload

open

THE SAME 46 LINES, AS ONE FILE AND AS THREE, AND WHERE A CHANGE LANDS

change request show the shelf name on every row

clear the highlight

ONE FILE — APP.JS

**9 to change · 47 to read**

one file, so there is no line you can be sure the change does not touch.  
app.js

```
1 import { SEED } from './items.js';
2
3 let items = SEED.map((item) => ({ ...item }));
4 let query = '';
5 let sort = 'name';
6
7 const list = document.querySelector('#list');
8 const count = document.querySelector('#count');
9 const queryField = document.querySelector('#query');
10 const sortField = document.querySelector('#sort');
11 const form = document.querySelector('#item-form');
12
13 function visible() {
14   return items
15     .filter((item) => item.title.includes(query))
```

THREE FILES

**10 to change · 19 to read**

a change to how a row LOOKS. It has one home. One file opened.  
state.js

```
1 let state = { items: [], query: '', sort: 'name' };
2 const listeners = new Set();
3
4 export const getState = () => state;
5
6 export function setState(patch) {
7   state = { ...state, ...patch };
8   for (const listener of listeners) listener(state);
9 }
10
11 export function selectVisible(state) {
12   return state.items
13     .filter((item) => item.title.includes(state.query))
14     .sort((a, b) =>
15       state.sort === 'number'
```

► Do not watch the “lines to change” number — it is nearly the same. Watch **LINES TO READ**.

# PERFORMANCE, HONESTLY

10 / 100 / 1000 / 5000 rows · measured on this machine, now

↺ reload

open ↗

WHAT A FULL REDRAW COSTS, AND WHAT ACTUALLY BREAKS FIRST

measure at 10 / 100 / 1000 / 5000 rows

list size

100 ▾

redraw this list

redraw narrowed to 5 rows

patch one row

TIMINGS, ON THIS MACHINE, JUST NOW

ROWS

FULL REDRAW

REDRAW + LAYOUT

PATCH ONE ROW

not measured yet

One frame at 60 Hz is 16.7 ms. Anything under it is invisible.

THE LIST

item 0

0

item 1

1

item 2

2

item 3

3

The text box is INSIDE the list and it redraws on every keystroke, which is exactly what a search box does. Type one character into it.

0

redraws that destroyed the focused element

0

redraws that threw away the scroll position. Scroll down first, then try each of the two redraw buttons — only one of them costs you the place you were.

WHERE THIS STOPS BEING FINE

a few hundred rows

fine

a control inside the redrawn region

not fine

a redraw on every keystroke over thousands of rows

not fine

an animation or a video inside the region

not fine

Only the first of those four is about speed. The fix for the other three is the same and it is not a framework: keep anything the user is holding on to OUTSIDE the part you replace.

► One frame at 60 Hz is 16.7 ms. **A FEW HUNDRED ROWS IS COMFORTABLY INSIDE IT**, and these numbers are this laptop's, not a blog post's.

## WHERE A FULL REDRAW STOPS BEING FINE

| SITUATION                                        | FINE? | THE FIX, AND IT IS NOT A FRAMEWORK        |
|--------------------------------------------------|-------|-------------------------------------------|
| a few hundred rows                               | yes   | —                                         |
| a control the user is holding, inside the region | no    | move the control OUT of the region        |
| a redraw per keystroke over thousands of rows    | no    | debounce, or render a window of the list  |
| an animation or a video inside the region        | no    | move it out, or replace only what changed |

**Only the first row is about speed.**

# THE FIVE MISTAKES, AND WHAT EACH ONE COSTS

| THE MISTAKE                             | HOW IT SHOWS UP                                                 |
|-----------------------------------------|-----------------------------------------------------------------|
| update the DOM and the array separately | a removed item comes back on the next add                       |
| store something derivable               | works perfectly until somebody changes the collection elsewhere |
| use the array index as identity         | changes the wrong item, and shows the right one changing        |
| a handler that draws                    | three things to remember per handler, forever                   |
| one long file with no layers            | every change means reading all of it                            |